

Fundamentals of Programming

Department of Computer & Software Engineering (DC&SE)
College of E&ME
NUST

Course Information

- Course Title: Fundamentals of Programming
- Course Code: CS-114
- Semester: Fall 2018

Books

□ Textbook:

- C++ How to program, P. Deitel and H. Deitel, Prentice Hall, Latest Issue

□ Reference:

- Object-Oriented Programming in C++, Robert Lafore, Sams Publishing, 4th edition, 2001
- Programming with ANSI C, DP Publications Ltd, 1995, By B. J. Holmes.
- The C programming language by Kernighan and Richie.
- Structured Computer Organization, Andrew S. Tanenbaum and Todd Austin, Prentice Hall, 6th edition, 2012

Course Contents

- Introduction to computers, central processing unit organization, main memory organization, secondary storage organization, input/output devices interaction/organization
- What is Programming? Algorithms, computer languages, Compiler, Assembler and Interpreter, Data and results.
- Data, Data types, data representation, identifiers, reserved words, variables, constants.
- Inputs and outputs, Standards Library, address operator, string I/O, character I/O.
- Escape sequences, format specifiers, assignment statement, Compound operators, Instruction Sequence, Sequential structures, ASCII.

Course Contents

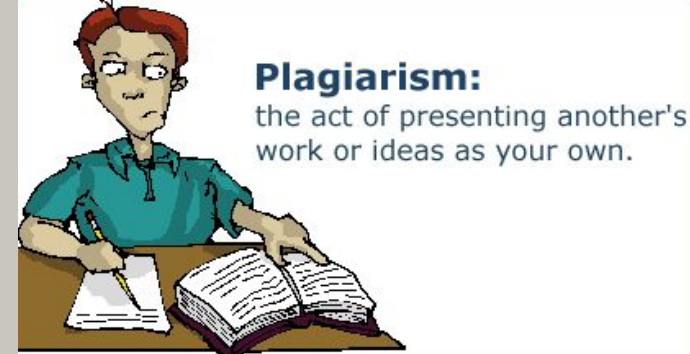
- Arithmetic operators, operator precedence, Selection, Relational and logical operators.
- If/else. Nested if's conditional operator, conditional expressions, switch statements.
- Repetition, While, do/while, for(;;), break and continue statements.
- Functions, Programmer defined functions, library functions, storage classes, scope, parameter passing.
- Arrays, input and output of data, searching.
- Array of characters, array as parameters.
- Pointers, address and indirection operators, pointer arithmetic, pointers and arrays, call by value and call by reference.

Course Learning Outcomes

Course Learning Outcomes (CLOs)	Level of Learning	PLO
1. Applying concepts of selection statements and repetition structures in solving complex problems.	C3	1
2. Understanding and implementing different functions in C++.	C3	1
3. Design algorithms to solve complex problems using arrays and pointers.	C6	3
4. Create a software development process by designing, implementing and testing C++ programs.	P2	3

Policies

- No extensions in assignment deadlines
- Quizzes will be unannounced.
- Exams will be closed book.
- Never cheat.
 - “Better fail NOW or else will fail somewhere LATER in life”
- Plagiarism will also have strict penalties.



Machine Languages, Assembly Languages, and High-level Languages

□ Three types of computer languages

1. Machine language

- Only language computer directly understands
- “Natural language” of computer
- Defined by hardware design
 - Machine-dependent
- Generally consist of strings of numbers
 - Ultimately 0s and 1s
- Instruct computers to perform elementary operations
 - One at a time
- Cumbersome for humans
- Example:

+1300042774

+1400593419

+1200274027

Machine Languages, Assembly Languages, and High-level Languages

□ Three types of computer languages

2. Assembly language

- English-like abbreviations representing elementary computer operations
- Clearer to humans
- Incomprehensible to computers
 - Translator programs (assemblers)
 - Convert to machine language
- Example:

```
LOAD    PURCHASE PRICE
ADD     SALES TAX
STORE   RETAIL PRICE
```

Machine Languages, Assembly Languages, and High-level Languages

□ Three types of computer languages

3. High-level languages

- Similar to everyday English, use common mathematical notations
- Single statements accomplish substantial tasks
 - Assembly language requires many instructions to accomplish simple tasks
- Translator programs (compilers)
 - Convert to machine language

- Example:

RETAIL PRICE = PURCHASE PRICE + SALES TAX

History of C and C++

□ History of C

- Evolved from two other programming languages
 - BCPL and B
- Dennis Ritchie (Bell Laboratories)
 - Added data typing, other features
- Development language of UNIX
- Hardware independent
 - Portable programs
- 1989: ANSI standard
- 1990: ANSI and ISO standard published

History of C and C++

□ History of C++

- Extension of C
- Early 1980s: Bjarne Stroustrup (Bell Laboratories)
- “Spruces up” C
- Provides capabilities for object-oriented programming
 - Objects: reusable software components
 - Model items in real world
 - Object-oriented programs
 - Easy to understand, correct and modify
- Hybrid language
 - C-like style
 - Object-oriented style
 - Both

C++ Standard Library

- Standardized version of C++
 - United States
 - American National Standards Institute (ANSI)
 - Worldwide
 - International Organization for Standardization (ISO)
- C++ programs
 - Built from pieces called classes and functions
- C++ standard library
 - Rich collections of existing classes and functions
- “Building block approach” to creating programs
 - “Software reuse”

Other High-level Languages

□ FORTRAN

- FORMula TRANslator
- 1954-1957: IBM
- Complex mathematical computations
 - Scientific and engineering applications

□ COBOL

- COmmon Business Oriented Language
- 1959: computer manufacturers, government and industrial computer users
- Precise and efficient manipulation of large amounts of data
 - Commercial applications

Other High-level Languages

□ Pascal

- Prof. Niklaus Wirth
- Academic use

Basics of a Typical C++ Environment

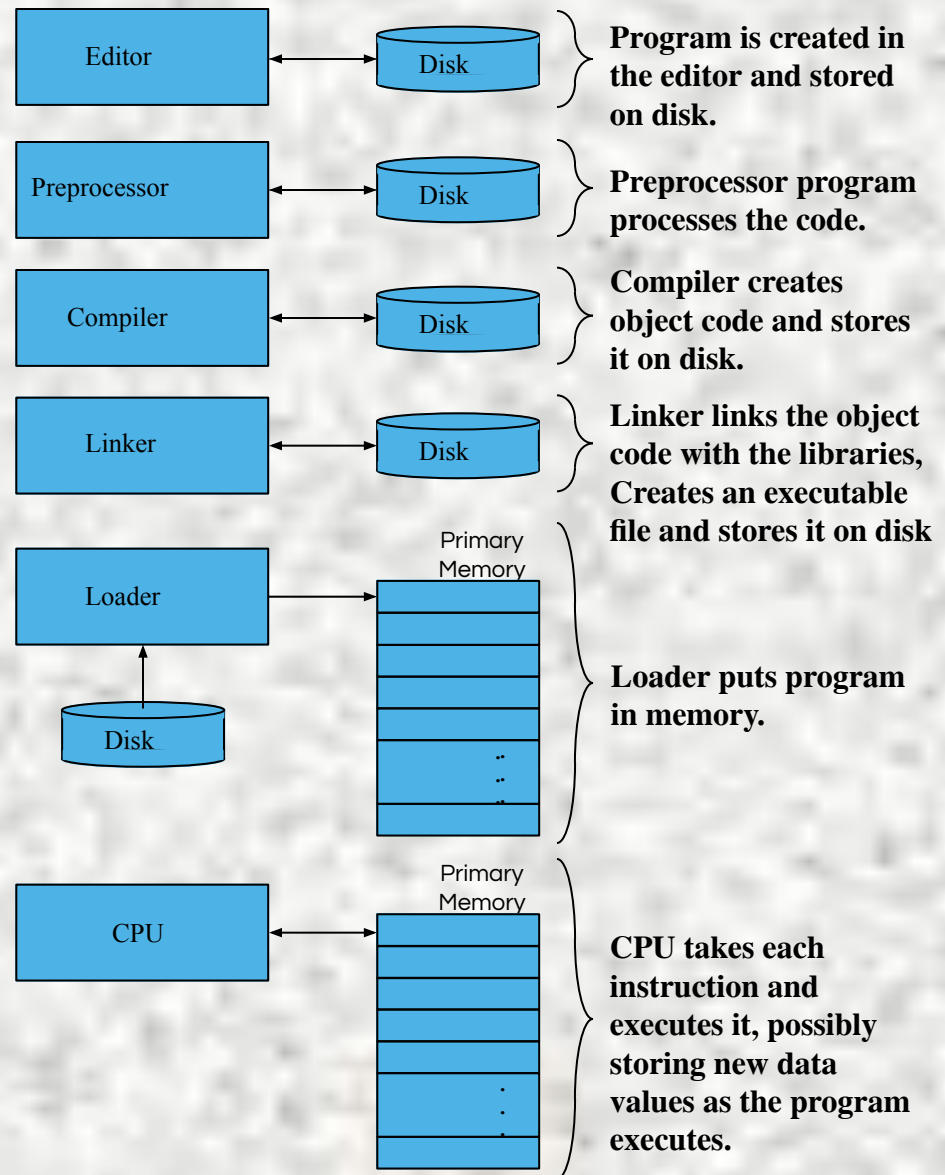
□ C++ systems

- Program-development environment
- Language
- C++ Standard Library

Basics of a Typical C++ Environment

Phases of C++ Programs:

1. Edit
2. Preprocess
3. Compile
4. Link
5. Load
6. Execute



A Simple Program: Printing a Line of Text

□ Input/output

– **cin**

- Standard input stream
- Normally keyboard

– **cout**

- Standard output stream
- Normally computer screen

A Simple Program: Printing a Line of Text

□ Comments

- Document programs
- Improve program readability
- Ignored by compiler
- Single-line comment
 - Begin with //

□ Preprocessor directives

- Processed by preprocessor before compiling
- Begin with #

A Simple Program: Printing a Line of Text

□ Standard output stream object

- “Connected” to screen
- <<
 - Stream insertion operator
 - Value to right (right operand) inserted into output stream

□ Escape characters

- \
- Indicates “special” character output


```
1 // A first program
```

Single-line comments.

```
2 #include <iostream>
```

Function **main** returns an

```
3 // function main begins
```

Left brace { begins function body.

```
4 int main()
```

```
5 {
```

occurs exactly once in every C++ program..

Statements end with a semicolon ;.

```
6 cout << "Welcome to
```

```
7 return 0; // ends function body successfully
```

```
8 }
```

Corresponding right brace }

```
9 return 0; // ends function body successfully
```

```
10 }
```

```
11 }
```

```
12 }
```

Stream insertion operator.

namespace **std**.

Keyword **return** is one of several means to exit function; value **0** indicates program terminated successfully.

Welcome to C++!

```
1
2 // Printing a line with multiple statements.
3 #include <iostream.h>
4
5 // function main begins program execution
6 int main()
7 {
8     cout << "Welcome ";
9     cout << "to C++!\n";
10
11     return 0;    // indicate that program ended successfully
12
13 } // end function main
```

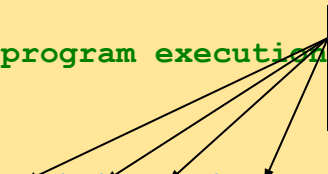
Multiple stream insertion statements produce one line of output.



```
Welcome to C++!
```

```
1
2 // Printing multiple lines with a single statement
3 #include <iostream.h>
4
5 // function main begins program execution
6 int main()
7 {
8     cout << "Welcome\nto\n\nC++!\n";
9
10     return 0;    // indicate that program ended successfully
11
12 } // end function main
```

Using newline characters to print on multiple lines.



```
Welcome
to

C++!
```

Another Simple Program: Adding Two Integers

□ Variables

- Location in memory where value can be stored
- Common data types
 - **int** - integer numbers
 - **char** - characters
 - **double** - floating point numbers
- Declare variables with name and data type before use

```
int integer1;  
int integer2;  
int sum;
```
- Can declare several variables of same type in one declaration
 - Comma-separated list

```
int integer1, integer2, sum;
```


Another Simple Program: Adding Two Integers

□ Variables

- Variable names
 - Valid identifier
 - Series of characters (letters, digits, underscores)
 - Cannot begin with digit
 - Case sensitive

Another Simple Program: Adding Two Integers

□ Input stream object

- >> (stream extraction operator)
 - Waits for user to input value, then press *Enter* (Return) key
 - Stores value in variable to right of operator
 - Converts value to variable data type

□ = (assignment operator)

- Assigns value to variable
- Binary operator (two operands)
- Example:

```
sum = variable1 + variable2;
```

```

1
2 // Addition program.
3 #include <iostream.h>
4
5 // function main begins program execution
6 int main()
7 {
8     int integer1; // first number to be input by user
9     int integer2; // second number to be input by user
10    int sum; // variable
11
12    cout << "Enter first integer\n"; // prompt
13    cin >> integer1; // read an integer
14
15    cout << "Enter second integer\n"; // prompt
16    cin >> integer2; // read an integer
17
18    sum = integer1 + integer2; // assign result to sum
19
20    cout << "Sum is " << sum << endl; // print sum
21
22    return 0; // indicate that program ended successfully
23
24 } // end function main

```

Declare integer variables.

Use stream extraction operator with standard input stream to obtain user input.

Stream manipulator **std::endl** outputs a newline, then “flushes output buffer.”

Concatenating, chaining or cascading stream insertion operations.

```
Enter first integer
45
Enter second integer
72
Sum is 117
```


Memory Concepts

□ Variable names

- Correspond to actual locations in computer's memory
- Every variable has name, type, size and value
- When new value placed into variable, overwrites previous value
- Reading variables from memory nondestructive

Memory Concepts

```
cin >> integer1;
```

- Assume user entered 45

integer1	45
----------	----

```
cin >> integer2;
```

- Assume user entered 72

integer1	45
----------	----

integer2	72
----------	----

```
sum = integer1 + integer2;
```

integer1	45
----------	----

integer2	72
----------	----

sum	117
-----	-----

Arithmetic

□ Arithmetic calculations

– *

- Multiplication

– /

- Division
- Integer division truncates remainder
 - $7 / 5$ evaluates to 1

– %

- Modulus operator returns remainder
 - $7 \% 5$ evaluates to 2

Arithmetic

□ Rules of operator precedence

- Operators in parentheses evaluated first
 - Nested/embedded parentheses
 - Operators in innermost pair first
- Multiplication, division, modulus applied next
 - Operators applied from left to right
- Addition, subtraction applied last

Operator(s)	Operation(s)	Order of evaluation (precedence)
()	Parentheses	Evaluated first. If the parentheses are nested, the expression in the innermost pair is evaluated first. If there are several pairs of parentheses “on the same level” (i.e., not nested), they are evaluated left to right.
*, /, or %	Multiplication Division Modulus	Evaluated second. If there are several, they re evaluated left to right.
+ or -	Addition Subtraction	Evaluated last. If there are several, they are evaluated left to right.

Decision Making: Equality and Relational Operators

□ **if** structure

- Make decision based on truth or falsity of condition
 - If condition met, body executed
 - Else, body not executed

□ Equality and relational operators

Decision Making: Equality and Relational Operators

Standard algebraic equality operator or relational operator	C++ equality or relational operator	Example of C++ condition	Meaning of C++ condition
<i>Relational operators</i>			
>	>	x > y	x is greater than y
<	<	x < y	x is less than y
≥	>=	x >= y	x is greater than or equal to y
≤	<=	x <= y	x is less than or equal to y
<i>Equality operators</i>			
=	==	x == y	x is equal to y
≠	!=	x != y	x is not equal to y

Decision Making: Equality and Relational Operators

```
□ if ( x == y )  
    cout << x << " is equal to " << y << endl;
```

```
1
2 // Using if statements, relational
3 // operators, and equality operators.
4 #include <iostream.h>
```

```
5
6
7
8
9
10 // function main begins program
```

Declare variables.

```
11 int main()
```

```
12 {
13     int num1; // first number
14     int num2; // second number
```

Can write **cout** and **cin** without **std::** prefix.

```
15
16     cout << "Enter two integers and I will tell you\n"
```

```
17     << "the relationships between them\n";
```

```
18     cin >> num1 >> num2; // read in the numbers
```

```
19
20     if ( num1 == num2 )
```

```
21         cout << num1 << " is equal to " << num2 << "\n";
```

```
22
23     if ( num1 != num2 )
```

```
24         cout << num1 << " is not equal to " << num2 << "\n";
```

```
25
```

if structure compares values

of **num1** and **num2** for equality.

If condition is true (i.e., values are equal), execute this statement.

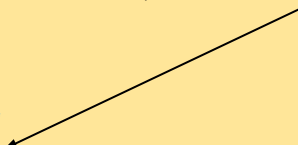
if structure compares values

of **num1** and **num2** for inequality.

If condition is true (i.e., values are not equal), execute this statement.

```
26  if ( num1 < num2 )
27      cout << num1 << " is less than " << num2 << endl;
28
29  if ( num1 > num2 )
30      cout << num1 << " is greater than " << num2 << endl;
31
32  if ( num1 <= num2 )
33      cout << num1 << " is less than or equal to "
34          << num2 << endl;
35
36  if ( num1 >= num2 )
37      cout << num1 << " is greater than or equal to "
38          << num2 << endl;
39
40  return 0;    // indicate that program ended successfully
41
42 } // end function main
```

Statements may be split over several lines.



```
Enter two integers, and I will tell you
the relationships they satisfy: 22 12
22 is not equal to 12
22 is greater than 12
22 is greater than or equal to 12
```

Enter two integers, and I will tell you
the relationships they satisfy: 7 7

7 is equal to 7

7 is less than or equal to 7

7 is greater than or equal to 7

Algorithms

- Before writing a program
 - Have a thorough understanding of problem
 - Carefully plan your approach for solving it

Algorithms

□ Computing problems

- Solved by executing a series of actions in a specific order

□ Algorithm a procedure determining

- Actions to be executed
- Order to be executed
- Example: recipe

□ Program control

- Specifies the order in which statements are executed

Pseudocode

□ Pseudocode

- Artificial, informal language used to develop algorithms
- Similar to everyday English

□ Not executed on computers

- Used to think out program before coding
 - Easy to convert into C++ program
- Only executable statements
 - No need to declare variables